EduGenie – Google Gemini Powered Learning Assistant

Project Description:

EduGenie is an AI-powered learning assistant designed to help students understand academic topics more effectively through Generative AI. The platform provides multiple learning features, including Question Answering, Topic Explanation, Quiz Generation, Notes Summarization, and Learning Recommendations. By simply entering a question or topic, students receive accurate, easy-to-understand, and personalized learning support in real time.

The application integrates Google Gemini API for intelligent content generation and Hugging Face's LaMini-Flan-T5 model for topic explanations. Built using FastAPI, HTML, CSS, JavaScript, and Jinja2, EduGenie offers a clean, responsive, and user-friendly web interface. Secure API key management is handled through a .env configuration file, ensuring safe and reliable access to AI services. EduGenie aims to improve students' learning experience by making complex concepts easier to understand while reducing the time spent searching across multiple resources.

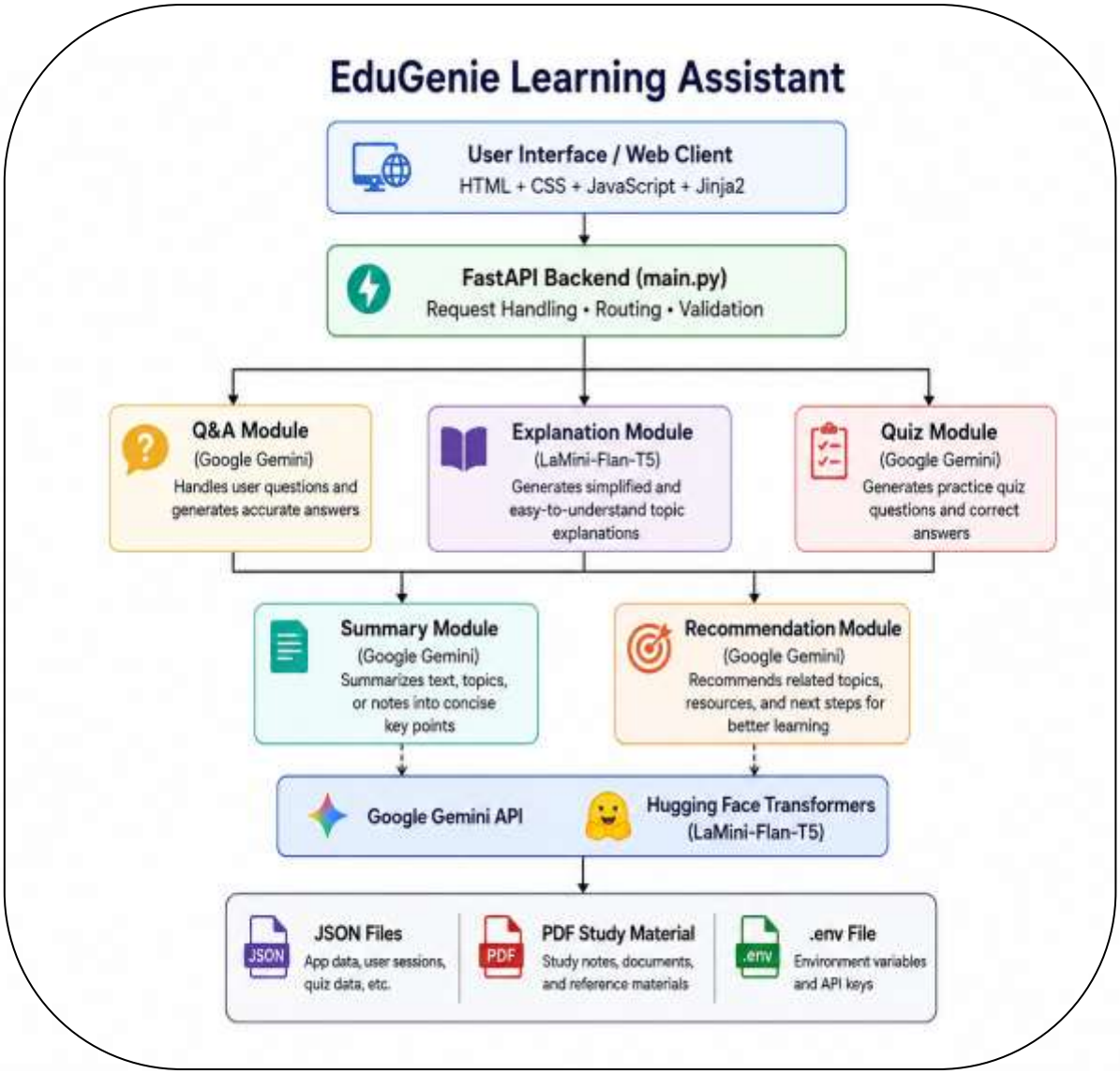
Scenarios

Scenario 1: A student preparing for an examination enters a difficult programming question into the Question Answering module. EduGenie instantly provides a clear and accurate explanation using the Google Gemini API, helping the student understand the concept quickly.

Scenario 2: A student studying a complex topic such as Operating Systems selects the Topic Explanation feature. EduGenie generates a simple, beginner-friendly explanation using the LaMini-Flan-T5 model, making the topic easier to learn.

Scenario 3: A student uploads study material before an exam and uses the Quiz Generation and Notes Summarization features. EduGenie creates concise notes, generates practice quiz questions, and recommends related learning topics, enabling effective revision in less time.

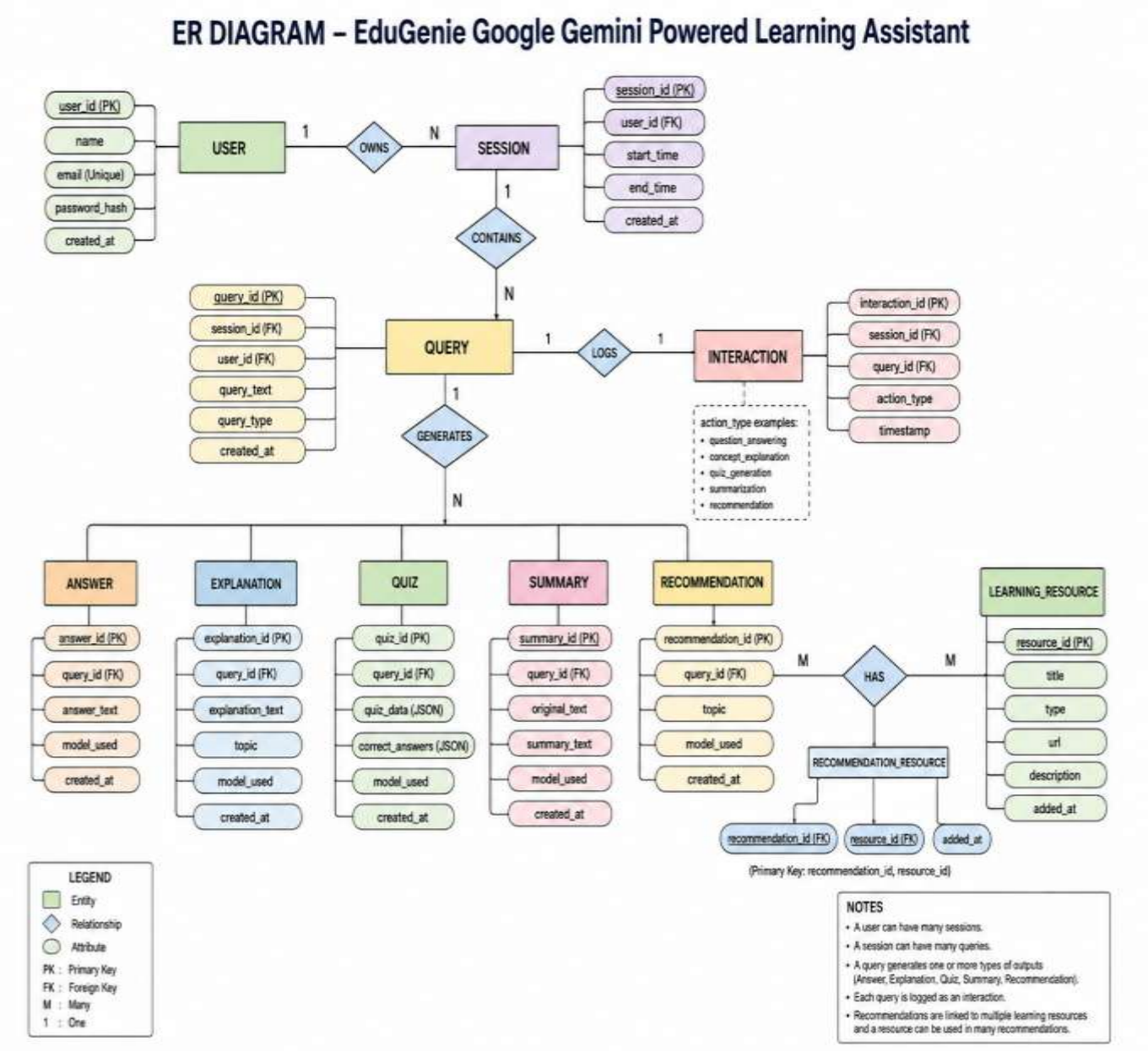
Technical Architecture:



Description:

EduGenie Learning Assistant follows a modular AI-powered architecture designed to help students learn more effectively. The frontend is developed using HTML, CSS, JavaScript, and Jinja2 templates to provide a simple web interface. User requests are handled by a FastAPI backend that validates inputs and routes them to the appropriate AI module. Google Gemini powers Question Answering, Quiz Generation, Notes Summarization, and Learning Recommendations, while the Hugging Face LaMini-Flan-T5 model generates simplified topic explanations. The application stores configuration using environment variables and uses local JSON files and PDF study materials without requiring a traditional database.

ER Diagram:



Description:

The ER Diagram (Entity Relationship Diagram) represents the logical data structure of the EduGenie Learning Assistant. It illustrates how users, sessions, queries, interactions, AI-generated responses (Question Answering, Explanation, Quiz, Summary, and Recommendation), and learning resources are connected. The diagram helps organize application data, improves maintainability, and provides a clear representation of relationships between different modules of the system

Pre-requisites:

- Python 3.10+
- FastAPI Framework
- Uvicorn Server
- HTML, CSS, JavaScript
- Jinja2 Templates
- Google Gemini API
- Hugging Face Transformers
- Git & GitHub
- VS Code
- Virtual Environment (venv)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Set up the Python virtual environment and install project dependencies (FastAPI, Uvicorn, Google GenAI SDK, Transformers, Jinja2, etc.).
- **Activity 1.2:** Select Google Gemini for Question Answering, Quiz Generation, Notes Summarization, and Learning Recommendations.
- **Activity 1.3:** Select Hugging Face LaMini-Flan-T5 for simplified Topic Explanation.
- **Activity 1.4:** Design the three-tier architecture consisting of the HTML/CSS/JavaScript frontend, FastAPI backend, and AI service modules.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the AI modules for Question Answering, Topic Explanation, Quiz Generation, Notes Summarization, and Learning Recommendations**.**
- **Activity 2.2:** Implement backend request routing and module integration using FastAPI.
- **Activity 2.3:** Configure API key management through .env and integrate Google Gemini and Hugging Face models.

Milestone 3: FastAPI Backend Development

- **Activity 3.1:** Develop main.py with API routes, request validation, and response handling.
- **Activity 3.2:** Connect all AI modules with the FastAPI backend and perform backend testing.

Milestone 4: Frontend Development

- **Activity 4.1:** Design the user interface using HTML, CSS, JavaScript, and Jinja2 templates.
- **Activity 4.2: Integrate the frontend with FastAPI to display AI-generated responses dynamically.**

Milestone 5: Testing and Deployment

- **Activity 5.1:** Perform functional testing for all EduGenie modules.
- **Activity 5.2:** Execute performance testing and verify application stability.
- **Activity 5.3:** Deploy and run the application locally using Uvicorn and prepare the GitHub repository and documentation for submission.

Milestone 1: Model Selection and Architecture

In this milestone, we establish the development environment, select the most suitable AI models for each learning task, and design the overall architecture of the EduGenie Learning Assistant. The goal is to ensure efficient integration between the frontend, backend, and AI services.

Activity 1.1: Set Up the Development Environment

Prepare the project environment before development.

- Install Python 3.10+ and required dependencies.
- Create and activate a virtual environment.
- Install all required libraries from `requirements.txt`.
- Configure the Google **Gemini API key** using the `.env` file.
- Verify the FastAPI server runs successfully.

Commands

- python -m venv venv
- venv\Scripts\activate
- pip install -r requirements.txt
- GOOGLE_API_KEY=your_api_key_here
- uvicorn main:app --reload

Activity 1.2: Research and Select AI Models

The project uses different AI models depending on the learning task.

- **Google Gemini** is selected for:
  - Question Answering
  - Quiz Generation
  - Notes Summarization
  - Learning Recommendations
- **LaMini-Flan-T5** is selected for:
  - Topic Explanation

These models were chosen for their balance of response quality, speed, and suitability for educational applications.

Activity 1.3: Define the Application Architecture

Design the overall system architecture showing communication between the frontend, backend, AI models, and supporting files.

The architecture includes:

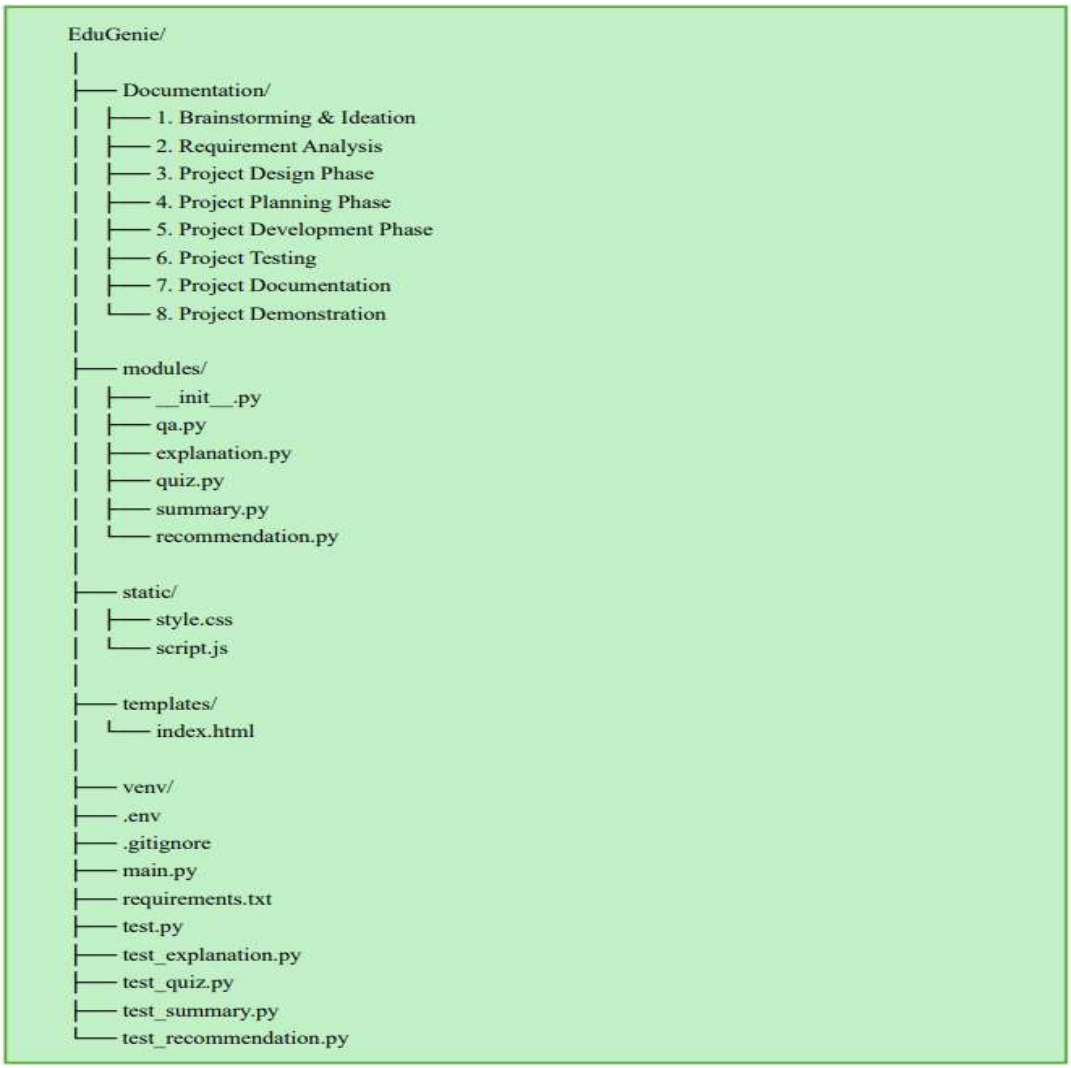
- **Frontend:** HTML, CSS, JavaScript, and Jinja2
- **Backend:** FastAPI
- **AI Modules:** Google Gemini and LaMini-Flan-T5
- **Configuration:** Environment variables (.env)
- **Resources:** JSON files and PDF study materials

Activity 1.4: Project Structure and Module Design

The project was organized into a modular folder structure to improve readability, maintainability, and scalability.

Main project components include:

- main.py – FastAPI application
- modules/ – AI functionality modules
- templates/ – Jinja2 HTML templates
- static/ – CSS and JavaScript files
- requirements.txt – Project dependencies
- .env – API configuration
- README.md – Project documentation



Milestone 2: Core Functionalities Development

This milestone focuses on implementing the core AI-powered learning features of EduGenie. Each module is designed independently and integrated with the FastAPI backend to provide an interactive learning experience.

Activity 2.1: Develop AI Modules

Develop separate AI modules to perform different educational tasks.

- **Question Answering Module:** Uses Google Gemini to answer academic questions accurately and contextually.
- **Topic Explanation Module:** Uses Hugging Face LaMini-Flan-T5 to simplify complex concepts into easy-to-understand explanations.
- **Quiz Generation Module:** Uses Google Gemini to generate topic-based quizzes with relevant questions.

- **Notes Summarization Module:** Produces concise summaries by extracting the key points from lengthy study material.
- **Learning Recommendation Module:** Recommends books, websites, courses, and learning resources based on the user's topic.

Activity 2.2: Implement AI Processing Logic

Develop the processing logic for each module.

- Accept user queries and selected learning features.
- Generate optimized prompts for the selected AI model.
- Process AI responses and format the output.
- Handle invalid or empty inputs using validation and exception handling.

Activity 2.3: Configure AI Model Integration

Integrate the required AI services into the project.

- Configure the Google Gemini API using the .env file.
- Integrate the Hugging Face LaMini-Flan-T5 model for explanations.
- Ensure each module communicates with its corresponding AI model.
- Verify that generated responses are returned successfully for further backend integration.

Milestone 3: FastAPI Backend Development

Milestone 3 focuses on developing the FastAPI backend that acts as the bridge between the frontend and the AI modules. The backend receives user requests, routes them to the appropriate AI module, validates inputs, and returns AI-generated responses.

Activity 3.1: Develop main.py and API Routes

Develop the main FastAPI application and create API endpoints for EduGenie.

- **Initialize FastAPI Application:** Create the FastAPI app and configure project settings.
- **Define API Routes:** Implement GET and POST endpoints to process user requests.
- **Request Validation:** Validate user inputs before sending them to AI modules.
- **Response Handling:** Return formatted AI-generated responses to the frontend using JSON.

Example Features:

- Home Route (/)
- Question Answering
- Topic Explanation
- Quiz Generation
- Notes Summarization
- Learning Recommendations

Activity 3.1: Integrate AI Modules with FastAPI

Connect all AI modules with the backend application.

- Import Question Answering, Explanation, Quiz, Summary, and Recommendation modules.
- Route each request based on the selected feature.
- Configure Google Gemini API and Hugging Face model loading.
- Handle API exceptions and invalid requests gracefully.
- Verify successful communication between backend and AI services.

Milestone 4: Frontend Development

Milestone 4 focuses on designing and developing the user interface of EduGenie. The frontend provides an interactive and user-friendly experience, allowing users to access AI-powered learning features and display results dynamically.

Activity 4.1: Design the User Interface

Develop the frontend using **HTML, CSS, JavaScript, and Jinja2 templates**.

- **Create the Homepage:** Design an intuitive homepage with a clean and responsive layout.
- **Build Input Forms:** Provide input fields for entering questions, topics, or study material.
- **Feature Selection:** Add options for Question Answering, Topic Explanation, Quiz Generation, Notes Summarization, and Learning Recommendations.
- **Responsive Design:** Ensure compatibility across desktop and mobile devices using CSS styling.

Activity 4.2: Integrate Frontend with FastAPI

Connect the frontend with the FastAPI backend to display AI-generated responses.

- **Send User Requests:** Submit user inputs to the appropriate FastAPI endpoint using HTML forms and JavaScript.
- **Dynamic Content Rendering:** Display AI-generated answers, explanations, quizzes, summaries, and recommendations without reloading the page.
- **Template Rendering:** Use Jinja2 templates to render responses dynamically.
- **Error Handling:** Display user-friendly messages for invalid inputs or API errors and ensure smooth interaction with the backend.

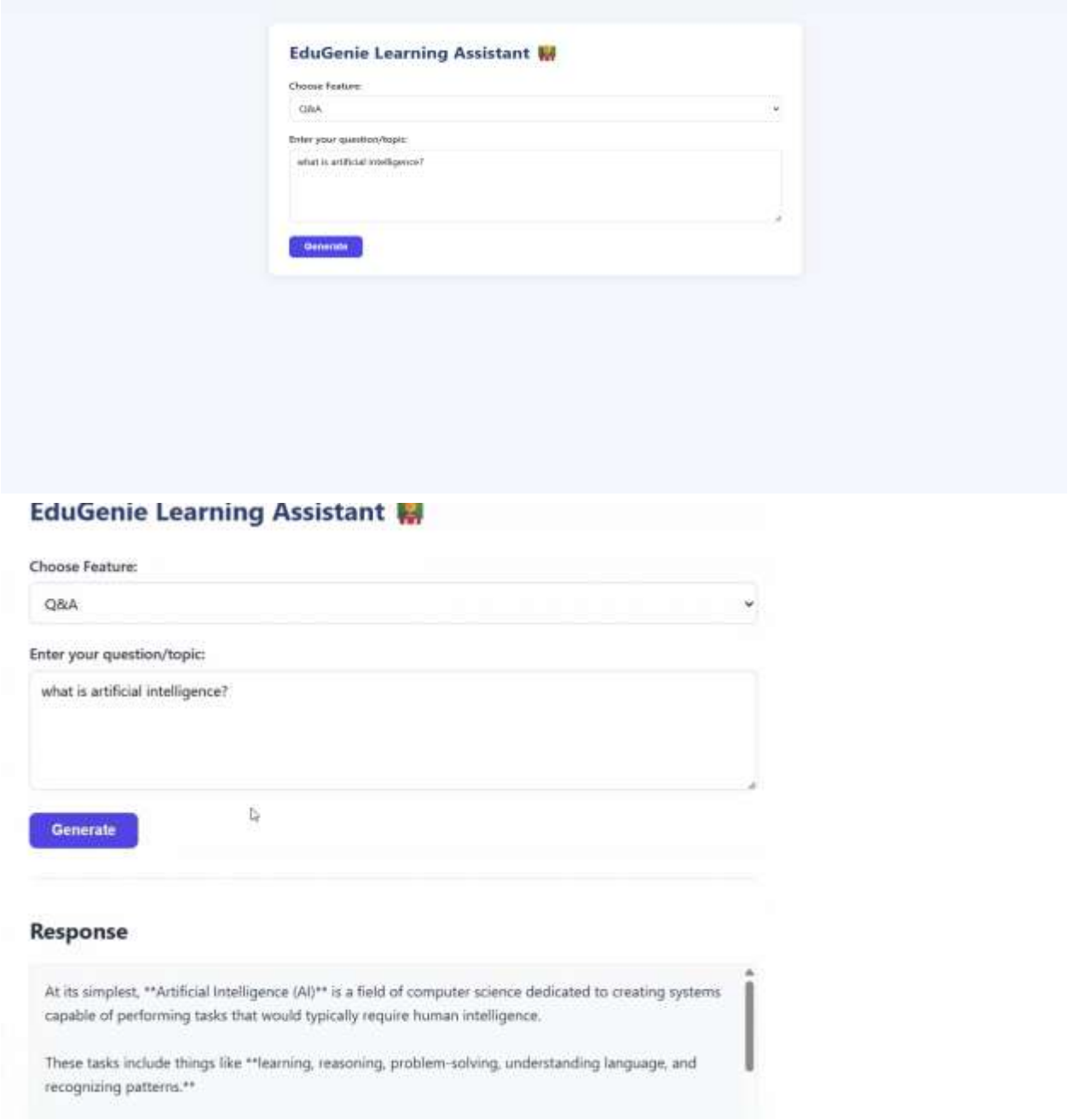
Milestone 5: Testing and Deployment

Milestone 5 focuses on validating the correctness, performance, and stability of the EduGenie Learning Assistant. After successful testing, the application is deployed locally and prepared for final submission.

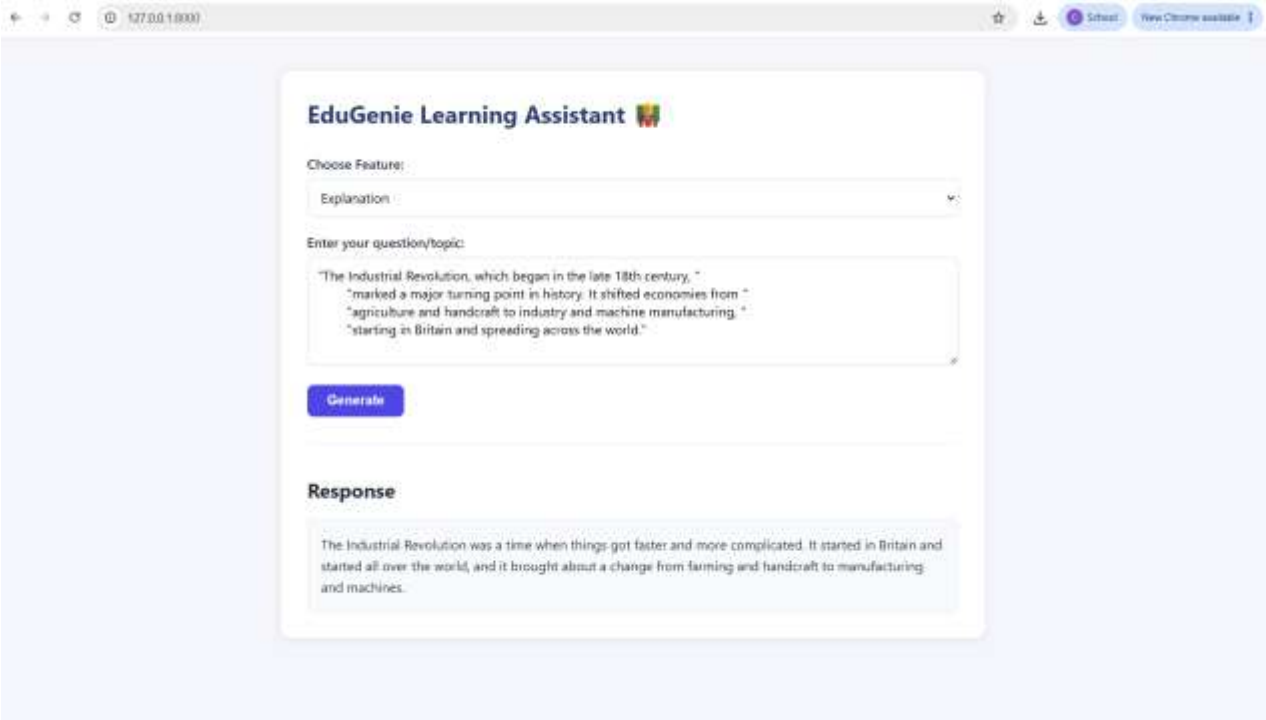
Activity 5.1: Functional Testing

Verify that each module of EduGenie works correctly.

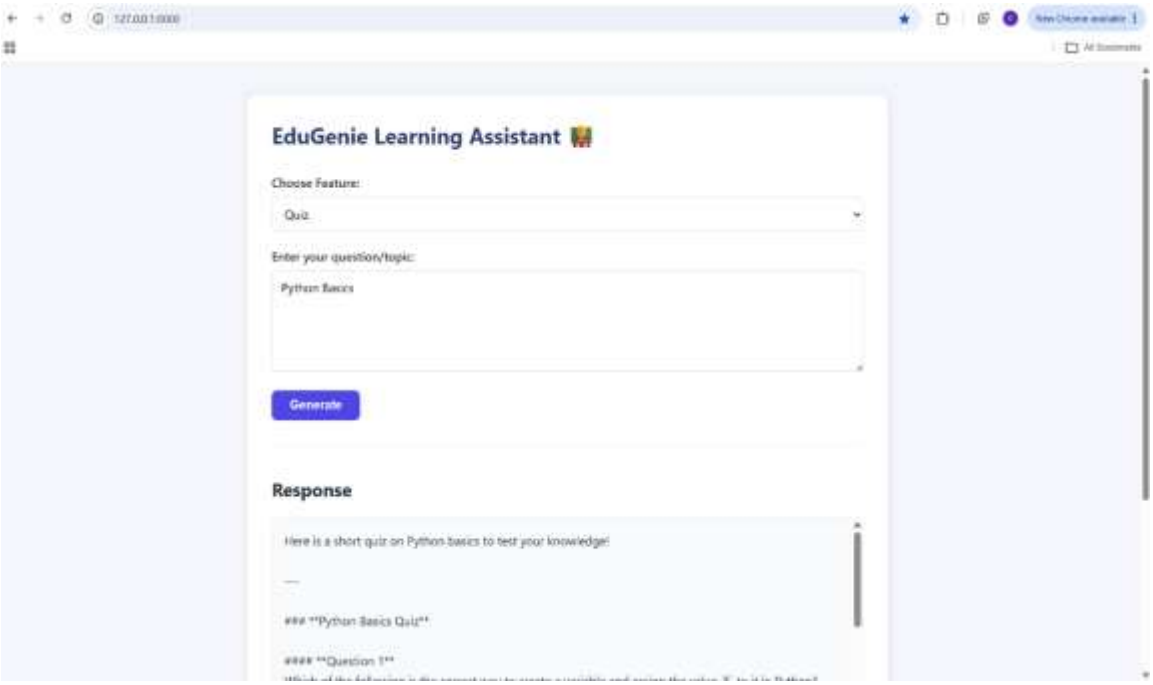
- Test **Question Answering** using different user questions.

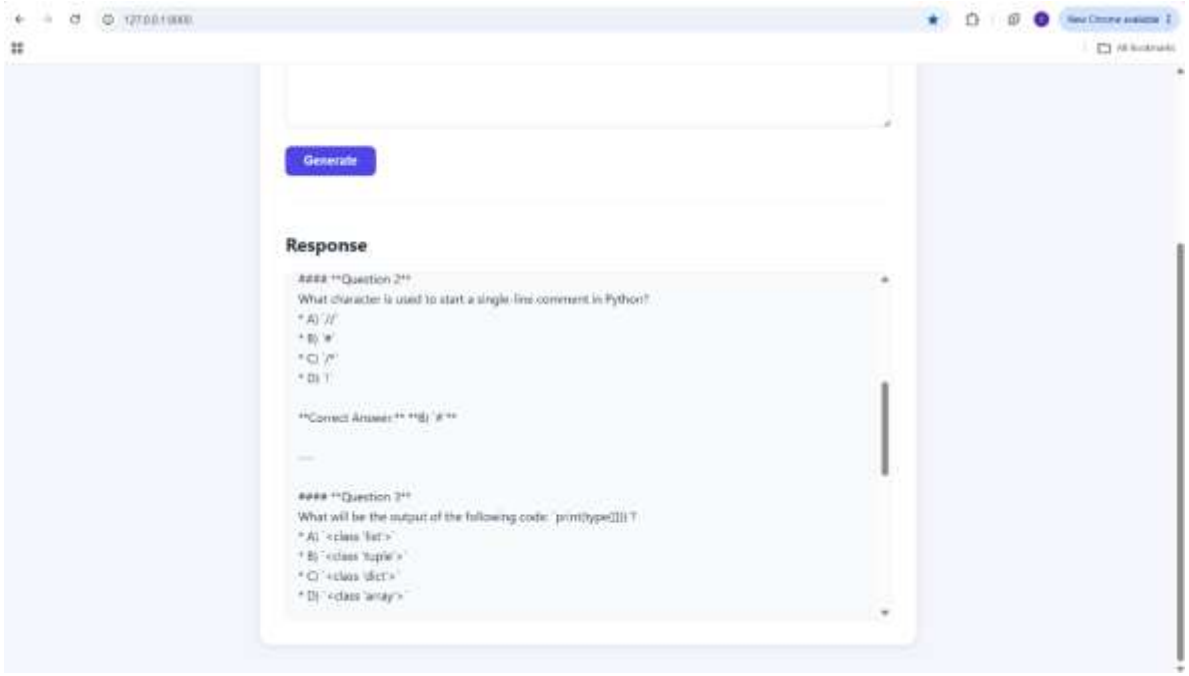


- Test **Topic Explanation** for simple and detailed explanations.

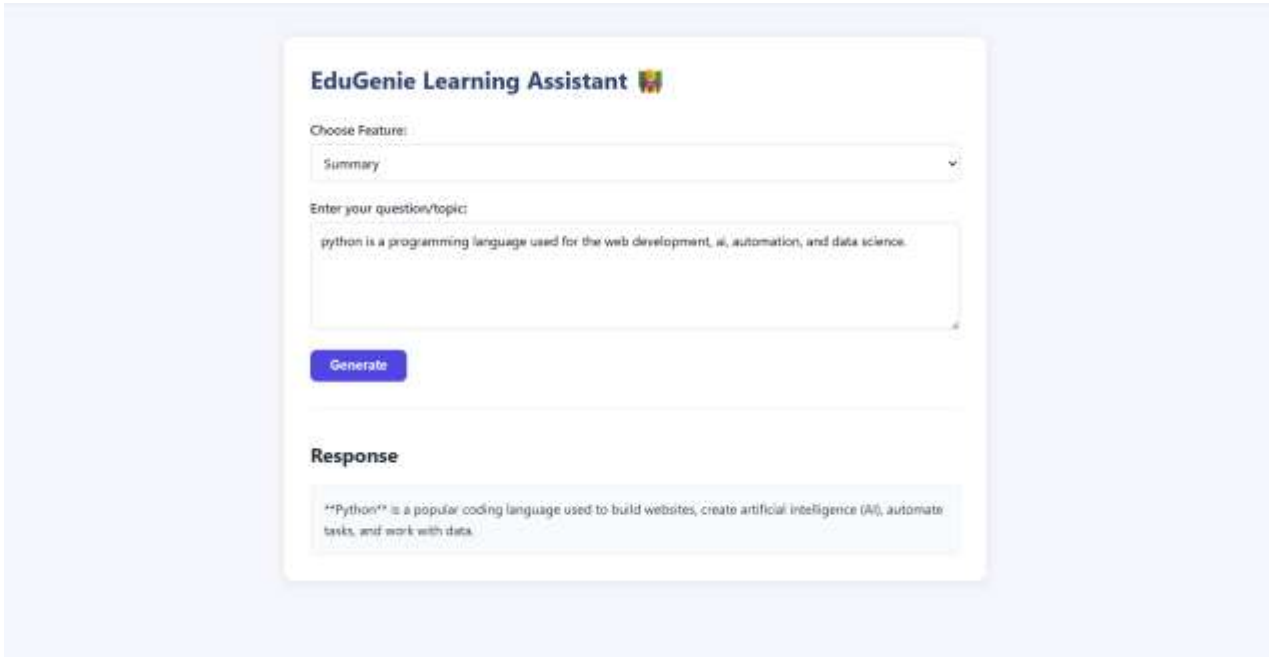


- Test **Quiz Generation** to ensure valid multiple-choice questions are generated.

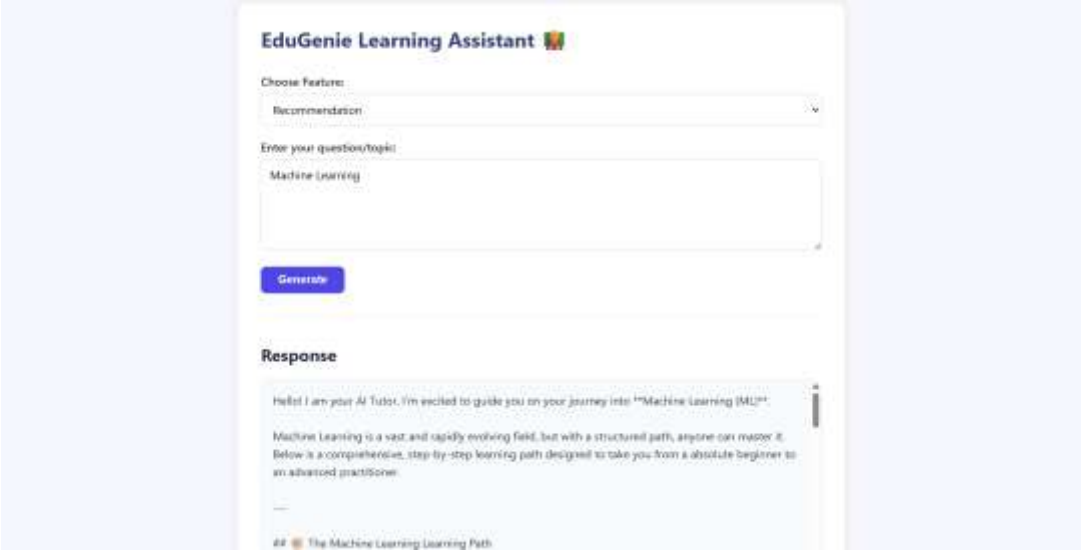




- Test **Notes Summarization** with lengthy study material.



- Test **Learning Recommendations** for relevant study resources.



Response

Recommended Books

1. **For Beginners:**
   * *"Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow"* by Aurélien Géron. (The absolute "bible" for practical ML).
2. **For Math & Theory (Intermediate):**
   * *"An Introduction to Statistical Learning" (ISLR)* by Gareth James et al. (Free PDF online, excellent balance of theory and application).
3. **For Deep Learning (Advanced):**
   * *"Deep Learning with Python"* by François Chollet (Creator of Keras. Incredibly intuitive explanations).

Recommended YouTube Channels

1. **[StatQuest with Josh Starmer](https://www.youtube.com/@statquest):** Broken down into simple,

- Verify input validation and error handling for invalid or empty requests.

Pretty-print ☐

{"detail":[{"type":"missing","loc":["body","query"],"msg":"Field required","input":null}]}

Activity 5.2: Performance Testing

Evaluate the application's performance under normal usage.

- Measure API response time.

```
uvicorn main:app --reload
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(venv) PS C:\Users\chait\OneDrive\Desktop\EduGenie> uvicorn main:app --reload
INFO:     Started server process [32716]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:64919 - "POST / HTTP/1.1" 200 OK
```

- Test multiple requests to verify backend stability.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(venv) PS C:\Users\chait\OneDrive\Desktop\EduGenie> uvicorn main:app --reload
INFO:     Started server process [32716]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:64919 - "POST / HTTP/1.1" 200 OK
INFO:     127.0.0.1:51490 - "POST / HTTP/1.1" 200 OK
INFO:     127.0.0.1:61173 - "POST / HTTP/1.1" 200 OK
INFO:     127.0.0.1:58355 - "GET / HTTP/1.1" 200 OK
INFO:     127.0.0.1:50770 - "POST / HTTP/1.1" 200 OK
INFO:     127.0.0.1:49667 - "GET / HTTP/1.1" 200 OK
INFO:     127.0.0.1:64180 - "GET / HTTP/1.1" 200 OK
```

- Monitor CPU and memory utilization.

  PU and memory usage remained stable during normal execution

- Ensure the application handles concurrent requests without failures.

The application was tested by sending multiple requests continuously. The FastAPI backend successfully processed all requests without failures or crashes, demonstrating stable concurrent request handling.

- Record testing observations using the Performance Testing document

Testing Observations:

- All EduGenie modules functioned correctly.
- API responses were generated successfully.
- Backend remained stable during repeated requests.
- CPU and memory usage remained within normal limits.
- No application crashes or unexpected errors were observed during testing.

Activity 5.3: Local Deployment and Project Submission

Prepare the project for execution and final submission.

- Activate the virtual environment.
- Install all dependencies from requirements.txt.
- Configure the .env file with the Google Gemini API key.
- Start the FastAPI server using Uvicorn.
- Verify that the application runs successfully in the browser.
- Push the latest project to GitHub.
- Upload documentation and demo video for SmartBridge submission.

Example commands:

```
python -m venv venv
venv\Scripts\activate

pip install -r requirements.txt

uvicorn main:app --reload
```

Conclusion:

The EduGenie Learning Assistant was successfully designed, developed, and tested as an AI-powered web application using FastAPI, HTML, CSS, JavaScript, Jinja2, Google Gemini API, and Hugging Face LaMini-Flan-T5. The application provides multiple learning features, including Question Answering, Topic Explanation, Quiz Generation, Notes Summarization, and Learning Recommendations through an intuitive user interface.

All project milestones, from architecture design and module development to backend integration, frontend implementation, testing, and local deployment, were completed successfully. Functional testing verified that each module produced accurate responses, while performance testing confirmed stable backend operation under normal usage.

This project demonstrates the practical application of Generative AI in education by helping learners understand concepts, generate quizzes, summarize study material, and receive personalized learning recommendations. The modular architecture also allows future enhancements such as user authentication, database integration, conversation history, multilingual support, and cloud deployment.

Overall, the EduGenie Learning Assistant fulfills the project objectives and serves as an effective AI-powered educational support system.